# 服务端如何处理并返回数据

- 定义前端请求的 url 规则 —— 路由
- 用 Request 获取数据，用 Response 返回数据
- 读取和存储数据

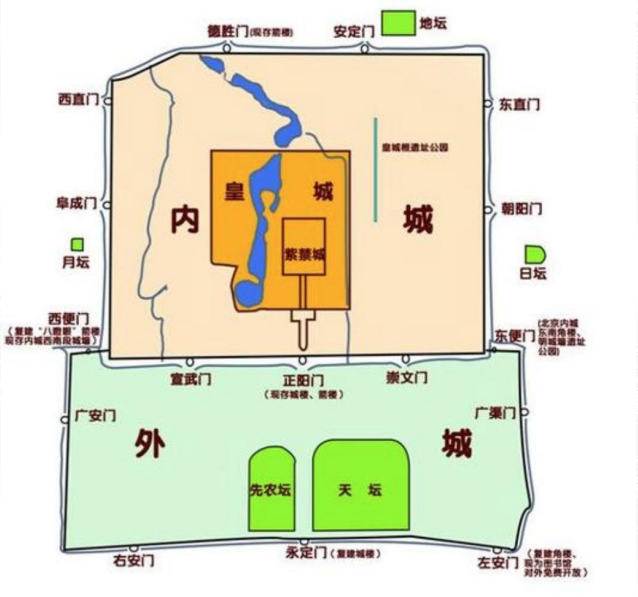
# 前端 Ajax 时需要 url

```
function getUserAccount() {
  return axios.get('/user/12345');
}

function getUserPermissions() {
  return axios.get('/user/12345/permissions');
}
```

# 路由是什么

- router
- 服务端的入口规则
- 和前端的约定

# 路由就像城门


德胜门(现存箭楼)
安定门
地坛
西直门
东直门
皇城根遗址公园
阜成门
朝阳门
内
皇
城
城
紫禁城
月坛
日坛
西便门
(复建"八里铺"箭楼
现存内城西南段城墙)
东便门
(北京内城
东南角楼、
明城墙遗址
公园)
宣武门
正阳门
(现存城楼、箭楼)
崇文门
广安门
广渠门
外
城
先农坛
天 坛
右安门
永定门(复建城楼)
左安门
(复建角楼、
现为邢台部
对外免费开放)

# 路由包含什么

- 定义 method，如 GET/POST 等
- 定义 url 规则，如 /api/list 和 /api/create
- 定义输入（Request body）和输出（Response body）格式

# 路由的代码示例

```
router.prefix('/api')

// 留言列表
router.get('/list', loginCheck, async function (ctx, next) {

    /* 此处省略 N 行 */

    ctx.body = {
        errno: 0,
        data: commentList
    }
})
```

# 路由的代码示例

```
router.prefix('/api')

// 创建留言
router.post('/create', loginCheck, async function (ctx, next) {
    const { content } =  ctx.request.body

    /* 次数省略 N 行 */

    ctx.body = {
        errno: 0,
        data: newComment
    }
})
```

# 路由和 url

- GET /api/list 路由 —> axios.get( ‘/api/list?a=1’ )
- POST /api/create 路由 —>axios.get( ‘/api/creaate’ , {…})
- 路由是规则，url 是具体的形式

# 路由 小结

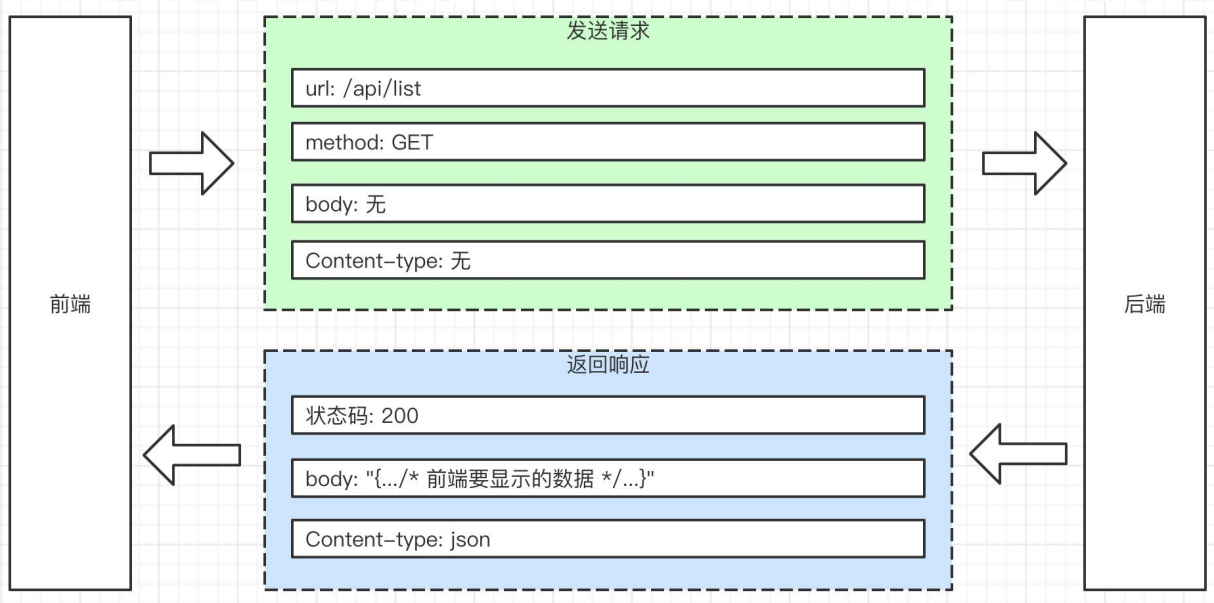
- 规定服务端的入口，入口即单个的功能
- 定义 method， url 规则，输入输出格式
- 路由是规则，url 是具体的形式，两者不同

# 分割线

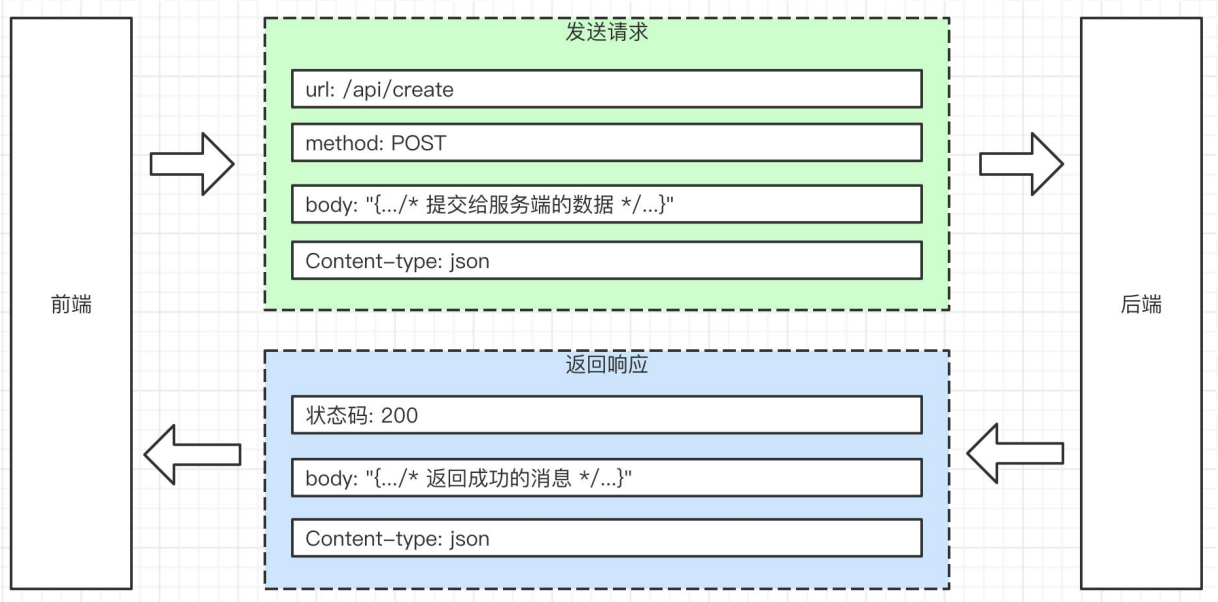
# 回顾 服务端的职责

- 提供前端要展示的数据
- 接收前端要提交的数据
- 存储数据（软件公司非常看重数据，会收集各种数据）

# 回顾 http GET 请求



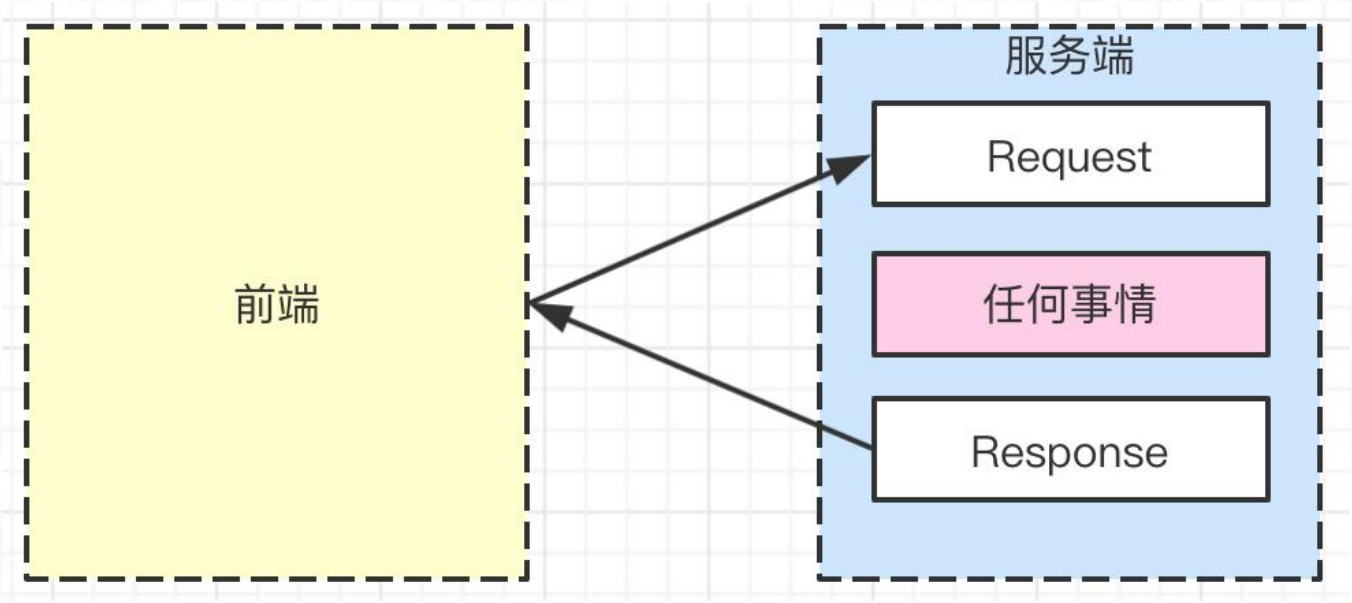
# 回顾 http POST 请求



# 服务端能得到 Request 和 Response

- 也可简称为 req 和 res
- 通过 Request 可获取：method url body
- 通过 Response: 可设置：状态码，Content-type，body

# 服务端拥有 Request 和 Response 即可做任何事



# 路由规则的实现离不开 Request

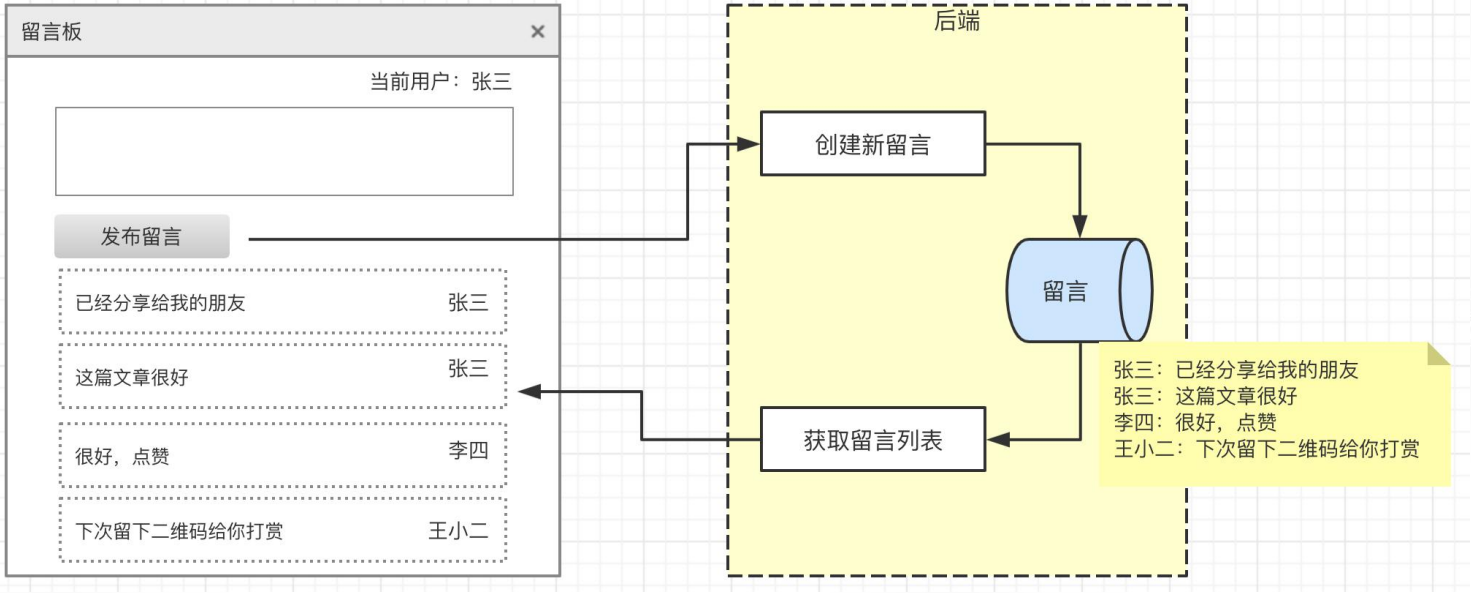
- 先定义路由规则（method、url 规则）
- 通过 Request 可获取：method url
- 检查 method 和 url 是否符合哪个路由

# 体验代码

```
router.prefix('/api')

// 创建留言
router.post('/create', loginCheck, async function (ctx, next) {
    const { content } =  ctx.request.body

    /* 次数省略 N 行 */

    ctx.body = {
        errno: 0,
        data: newComment
    }
})
```

# 分割线

# 回顾留言板前后端交互


留言板
当前用户：张三
发布留言
已经分享给我的朋友 张三
这篇文章很好 张三
很好，点赞 李四
下次留下二维码给你打赏 王小二
后端
创建新留言
留言
获取留言列表
张三：已经分享给我的朋友
张三：这篇文章很好
李四：很好，点赞
王小二：下次留下二维码给你打赏

# 回顾留言板前后端交互

- 获取留言板列表，数据从哪里来？
- 提交留言，数据存储到哪里？
- 服务器重启或者宕机，数据会不会丢失？

# 数据库

- 数据库，专职做数据的存储和查询
- 数据库是一个独立的系统，不是 nodejs 专属的
- 先不管数据库是什么，就先以为是一个 JSON

# 先把数据库当做是 JSON

```
const data = [
    { _id: '4', user: '张三', content: '已经分享给我的朋友', createdAt: '2020-03-30' },
    { _id: '3',  user: '张三', content: '这篇文章很好', createdAt: '2020-03-29' },
    { _id: '2',  user: '李四', content: '很好，点赞', createdAt: '2020-03-20' },
    { _id: '1',  user: '王小二', content: '下次留下二维码给你打赏', createdAt: '2020-03-15' }
]
```

# 数据库基本操作

- 新增
- 删除
- 修改
- 查询
- 查询条件
- 排序

# 新增一条留言

```
const data = [
    { _id: '5', user: '张三', content: '又来点赞', createdAt: '2020-04-01' },
    { _id: '4', user: '张三', content: '已经分享给我的朋友', createdAt: '2020-03-30' },
    { _id: '3',  user: '张三', content: '这篇文章很好', createdAt: '2020-03-29' },
    { _id: '2',  user: '李四', content: '很好，点赞', createdAt: '2020-03-20' },
    { _id: '1',  user: '王小二', content: '下次留下二维码给你打赏', createdAt: '2020-03-15' }
]
```

# 删除一条留言

```
const data = [
    { _id: '5', user: '张三', content: '又来点赞', createdAt: '2020-04-01' },
    { _id: '4', user: '张三', content: '已经分享给我的朋友', createdAt: '2020-03-30' },
    { _id: '3',  user: '张三', content: '这篇文章很好', createdAt: '2020-03-29' },
    { _id: '2',  user: '李四', content: '很好，点赞', createdAt: '2020-03-20' },
    { _id: '1',  user: '王小二', content: '下次留下二维码给你打赏', createdAt: '2020-03-15' }
]
```

删除时需要有一个条件，例如 _id === '3'

# 更新一条留言

```
const data = [
    { _id: '5', user: '张三', content: '哈哈', createdAt: '2020-04-01' },
    { _id: '4', user: '张三', content: '已经分享给我的朋友', createdAt: '2020-03-30' },
    { _id: '2',  user: '李四', content: '很好，点赞', createdAt: '2020-03-20' },
    { _id: '1',  user: '王小二', content: '下次留下二维码给你打赏', createdAt: '2020-03-15' }
]
```

更新时需要条件和更新内容，如条件是 _id === ‘5’，更新内容是 content: ‘哈哈’

# 查询留言列表

```
const data = [
    { _id: '5', user: '张三', content: '哈哈', createdAt: '2020-04-01' },
    { _id: '4', user: '张三', content: '已经分享给我的朋友', createdAt: '2020-03-30' },
    { _id: '2',  user: '李四', content: '很好，点赞', createdAt: '2020-03-20' },
    { _id: '1',  user: '王小二', content: '下次留下二维码给你打赏', createdAt: '2020-03-15' }
]
```

可设置排序，如 createdAt 逆序；可设置筛选条件，如 user === ‘张三’

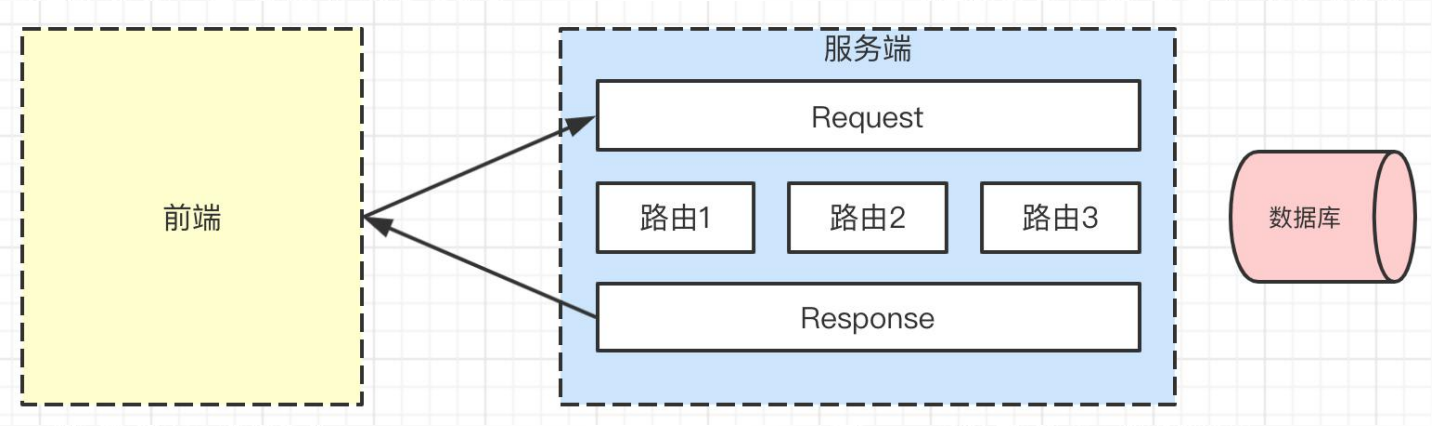
# 数据库 小结

- 负责数据存储和查询
- 先当做一个 JSON 来理解
- 基本操作：增、删、改、查

# 服务端处理数据 小结

- 定义前端请求的 url 规则 —— 路由
- 用 Request 获取数据，用 Response 返回数据
- 读取和存储数据 —— 数据库

# 服务端处理数据 小结



# 接下来讲解，为什么选择 nodejs